

Lingual-Fusion Adapter-Based Transfer Learning for Low-Resource Code Vulnerability Detection

Xinyue Long, Shikai Guo*, Yu Chai*, Hui Li, Sumaira Ameer Jan, Qian Ma, Qiao Ning

Abstract—Software vulnerabilities pose significant security threats to modern systems, particularly those involving complex execution sequences and intricate call relationships across multiple execution points. For instance, in a scenario where a software system integrates legacy code in a low-resource programming language like PHP, detecting vulnerabilities becomes challenging due to data scarcity and the complexity of temporal relationships among code fragments. This scarcity hampers the ability to capture critical temporal features essential for identifying vulnerabilities spanning multiple execution points. Consequently, existing approaches face major limitations, including neglecting temporal information in code fragments and lacking sufficient data to enable effective generalization for models in low-resource languages. To address these challenges, we introduce TaVer, a novel approach that enhances vulnerability detection in low-resource languages by extracting complex temporal features from code fragments and employing parameter-efficient transfer learning to leverage shared knowledge from resource-rich languages. TaVer comprises two key components: 1) Code Vulnerability Detection Component: This component models temporal dependencies by leveraging execution paths extracted from Abstract Syntax Trees (ASTs), capturing both short-term variations and long-term dependencies among code fragments. This enables comprehensive extraction of complex temporal features, significantly enhancing the accuracy of vulnerability detection. 2) Cross-Lingual Transfer Component: This component learns generalizable features from resource-rich languages and efficiently transfers them to low-resource languages. By updating a small number of downstream parameters, it enhances model generalization and achieves precise vulnerability detection. We evaluated TaVer using a diverse set of programming languages from publicly available GitHub repositories, employing C as the resource-rich source language and Java, Python, and PHP as relatively low-resource target languages. Experimental results demonstrate that TaVer outperforms four state-of-the-art approaches across multiple low-resource languages. Specifically, TaVer achieves average improvements of 14.63% in Accuracy, 30.59% in Precision, 37.32% in Recall, and 33.65% in F1-Score score over the best baseline approaches.

Index Terms—Code Vulnerability Detection, Transfer Learning, Multi-Language Environments, Open Source

1 INTRODUCTION

WITH the widespread application of software systems across various fields, the security risks posed by software vulnerabilities have become increasingly prominent. In recent years, the frequency of cybersecurity incidents, such as ransomware attacks and user data breaches, has further exacerbated threats to information security [1]. The number of reported Common Vulnerabilities and Exposures (CVEs) globally has surged rapidly since 2016, exceeding 25,000 annually in both 2022 and 2023 (as shown in Figure Fig. 1) [2]. Additionally, the 2020 Open Source Security and Risk Analysis (OSSRA) report published by Synopsys [3] highlights that a significant number of commercial codebases contain high-risk open-source vulnerabilities.

In the face of this increasingly severe security landscape, efficient detection and mitigation of software vulnerabilities have become critical challenges. Existing approaches can be broadly classified into pattern-based and code similarity-

based detection techniques. Traditional pattern-based approaches [4], [5], [6], [7] analyze code using predefined rules and demonstrate certain advantages in detecting specific vulnerabilities (e.g., memory-related issues). However, these approaches heavily rely on expert knowledge and struggle to handle the growing complexity of modern software systems [8]. In contrast, code similarity-based approaches [9], [10], [11] leverage data mining and machine learning techniques to predict vulnerabilities through code snippet similarity, eliminating the need for manual feature engineering. This has made them an increasingly promising solution.

Recent advancements in deep learning have significantly advanced the automation of code vulnerability detection [12], [13], [14], [15], [16], [17], [18], [19]. These approaches reduce dependence on manual feature engineering through automatic feature extraction. For instance, the VulDeePecker approach proposed by Li et al. [13] employs program slicing techniques and recurrent neural networks (RNNs) for vulnerability detection. Zhou et al. [15] utilize graph neural networks (GNNs) to construct composite graphs from abstract syntax trees (ASTs), control flow graphs (CFGs), and data flow graphs (DFGs) to identify vulnerabilities. Additionally, cross-domain anomaly detection research has introduced new perspectives to vulnerability detection. Studies in areas such as e-commerce, industrial anomaly detection, and fake news detection have demonstrated the successful application of cross-domain techniques and deep learning approaches [20], [21], [22]. These advancements

- X. Long and S. Guo are with the School of Information Science and Technology, Dalian Maritime University, Dalian, China, and the Key Laboratory of Artificial Intelligence of Dalian, Dalian, China. E-mail: xinyue.long@dlmu.edu.cn, shikai.guo@dlmu.edu.cn.
- Y. Chai is with Shanghai Jiao Tong University, Shanghai, China. E-mail: cy7372@sjtu.edu.cn
- H. Li, S. Jan, Q. Ma and Q. Ning are with the school of Information and Science Technology, Dalian Maritime University, Dalian, China. E-mail: li_hui@dlmu.edu.cn, sumairaameer-jan@gmail.com, maqian@dlmu.edu.cn, ningq669@dlmu.edu.cn.

* Corresponding author

underscore the critical role of advanced detection models in addressing cross-domain security challenges. Furthermore, techniques such as multi-hypergraph encoders for complex relationship modeling [23] and self-supervised multi-source heterogeneous data fusion [24] provide valuable insights for integrating multi-source features effectively in vulnerability detection.

However, as software systems grow increasingly complex, the execution order and invocation relationships between code fragments (i.e., implicit temporal relationships) play a critical role in code vulnerability detection. These temporal relationships reflect the dynamic runtime behavior of code, including dependencies, logic flows, and conditional judgments among different code fragments [25]. Accurately capturing these implicit temporal patterns is essential for comprehensively understanding the potential vulnerability risks during code execution. However, existing approaches exhibit significant limitations in modeling complex temporal relationships. They often focus solely on static or local dependencies of code, failing to capture the global temporal dependencies across multiple code fragments. Consequently, the extracted features for vulnerability detection lack comprehensiveness. Under conditions of scarce data resources, the limitations of these incomplete features are further amplified, adversely affecting the performance of transfer learning. In other words, the combined effects of incomplete features and data scarcity lead to a significant decline in detection performance, making it difficult to meet practical requirements.

To achieve more precise code vulnerability detection, the following two major challenges must be addressed:

Neglecting Temporal Information in Code Execution Paths. Code execution paths involve interactions across multiple time points. Existing approaches, such as those based on RNNs or CNNs, face challenges in handling long-term dependencies and complex temporal patterns [13], [14], [19], [26]. For example, the EPVD approach proposed by Zhang et al. [26] represents an execution path using the feature vector of its first time step and employs CNNs to capture inter-path attention. However, it overlooks critical features in subsequent time steps, hindering its ability to model complex temporal patterns and long-term dependencies comprehensively. This limitation is particularly pronounced in large-scale software systems, where execution paths are numerous and contain critical vulnerability information. A single model often struggles to effectively capture multi-scale temporal variations, thereby affecting detection accuracy and efficiency.

Lacking Sufficient Data for Effective Model Generalization. In real-world code vulnerability detection scenarios, low-resource programming languages face challenges in generalization due to the reliance of traditional deep learning models on large labeled datasets [12], [13], [16], [17]. For instance, existing models such as SySeVR [12] and Reveal [16] are primarily trained on data from resource-rich programming languages like C/C++, which feature abundant and well-labeled data, enabling high training accuracy. However, when these models are applied to low-resource or emerging languages with limited data availability and high labeling costs, their performance often deteriorates significantly. Moreover, syntactic and semantic differences across

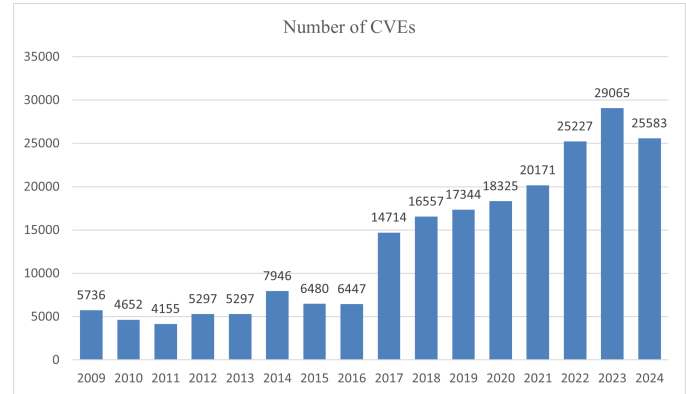


Fig. 1: Number of common IT security vulnerabilities and exposures (CVEs) worldwide from 2009 to 2024 YTD.

programming languages further increase the difficulty of cross-language transfer. Models that perform well in source languages often fail to achieve similar effectiveness in target languages.

To overcome the challenges outlined above, we propose a novel approach named **TaVer**, which combines Temporal analysis and Lingual-Fusion Adapter-based Transfer Learning for Low Resource code Vulnerability Detection. This approach comprises two components, each addressing one of these challenges:

Specifically, the Code Vulnerability Detection Component is designed to analyze and detect potential vulnerabilities within code. TaVer first parses the code into an Abstract Syntax Tree (AST) and subsequently generates a Control Flow Graph (CFG). In the CFG, each node represents a statement, and each edge represents a control dependency between two statements. Using a greedy path selection algorithm, representative code execution paths are chosen from the entry node to an exit node. These paths capture vulnerability-related information within the code fragments under analysis. The selected execution paths are then fed into the pre-trained CodeBERT model to capture intra-path attention and learn initial feature vectors. To better model the interactions across multiple time points within the code execution paths, the one-dimensional path feature sequences are transformed into two-dimensional tensors. This transformation enables the capture of both short-term variations within individual paths and long-term dependencies across different paths. By combining CNN-based inter-path attention mechanisms, this approach enhances the learning of complex temporal information, significantly improving the accuracy of code vulnerability detection.

Next, the Cross-Lingual Transfer Component, designed to address the challenge of data scarcity and limited generalization in code vulnerability detection for low-resource programming languages. TaVer achieves accurate and efficient detection under limited data conditions by learning shared patterns from resource-rich languages and transferring them to low-resource tasks. Specifically, this component first performs full-parameter training on datasets of resource-rich programming languages to learn general vulnerability-related features. Subsequently, when handling low-resource language data, only a small number of downstream param-

eters related to the target language are updated based on the pre-learned general features. This approach effectively reduces the risk of overfitting while enhancing the model's adaptability in multilingual environments, enabling efficient and precise vulnerability detection.

We conducted comprehensive experiments on multiple programming language datasets sourced from publicly available GitHub repositories. Specifically, we used the C-language dataset from D2A [27] as a resource-rich source language, and Java, Python, and PHP datasets from CrossVul [28] as comparatively low-resource target languages to evaluate the effectiveness of TaVer. We compared TaVer with state-of-the-art code vulnerability detection approaches [26], [29], [30], [31]. The experimental results show that TaVer achieves an average Precision of 41.79%, which is 30.59% higher than the best-performing baseline.

In summary, the main contributions of this paper are as follows:

- We propose a novel approach for code vulnerability detection, named TaVer, which integrates temporal analysis with cross-lingual transfer learning, making it particularly suited for resource-constrained scenarios. The Code Vulnerability Detection Component effectively captures complex temporal patterns within code execution paths, addressing the limitations of traditional approaches in handling temporal information during vulnerability detection. Meanwhile, the Cross-Lingual Transfer Component facilitates efficient knowledge transfer across programming languages, effectively overcoming the challenges of model generalization in low-resource environments.
- We conducted experiments on a public dataset to validate the effectiveness of the TaVer approach. The experimental results show that the TaVer approach outperforms the state-of-the-art models in Precision by 30.59%, 120.18%, 63.05%, and 43.41%, and in F1-Score by 33.66%, 70.56%, 84.82%, and 34.33%. Furthermore, Accuracy improved by 14.63%, 124.82%, 17.71%, and 25.44%.
- We make our code and experimental datasets publicly available as open-source to facilitate the advancement and development of research in code vulnerability detection and related fields [32].

The rest of this paper is structured as follows. Section 2 reviews the related work. Section 3 introduces the TaVer approach and provides detailed descriptions of its main components. In Section 4 and 5 we describe the experimental setup and presents the results, respectively. We then discuss the potential threats to validity in Section 6. Finally, Section 7 concludes this study and outlines directions for future research.

2 RELATED WORK

Early approaches to code vulnerability detection predominantly relied on manually crafted rules and features to identify potential vulnerabilities. Tools like FlawFinder [4], RATS [5], Cppcheck [6], and Checkmarx [7] employed pre-defined patterns to scan code for known issues. While effective in certain cases, these approaches were labor-intensive and highly dependent on human expertise, limiting their

scalability and adaptability to rapidly evolving codebases and large open-source projects.

To address these limitations, machine learning-based approaches were introduced, leveraging manually engineered features as inputs to predictive models [33], [34], [35]. Metrics such as code complexity, function calls [33], developer activity [35], API symbols [36], and subtree patterns [37] were employed to represent code vulnerabilities. However, these approaches also required significant domain knowledge and were slow to adapt to new codebases and emerging vulnerabilities.

With the rapid advancement of deep learning, researchers began exploring approaches for automatically learning semantic features from code [14], [19], reducing the dependence on manual feature engineering. Some approaches [13], [14] treat source code as a sequence of tokens and apply deep learning architectures inspired by natural language processing (NLP) for vulnerability detection. For example, Russell et al. [14] used recurrent neural networks (RNNs) to extract features from sequences of code tokens, while Dam et al. [19] proposed a Long Short-Term Memory (LSTM)-based model to capture the semantic and syntactic features of code. However, these sequence-based models often overlooked structural information within the code, such as abstract syntax trees (ASTs), leading to reduced detection accuracy.

To better capture structural information, graph-based models were introduced, representing code with various graph structures and using neural networks to learn the structural properties of code fragments for vulnerability detection [15], [16], [30], [38], [39], [40], [41]. Graph Neural Networks (GNNs) became a prominent approach, embedding nodes and graphs into low-dimensional continuous vector spaces and demonstrating strong performance in learning complex code structures [42], [43], [44]. For instance, Zhou et al. [15] applied GNNs to capture structural information from ASTs, control flow graphs (CFGs), and data flow graphs (DFGs) for vulnerability detection. Similarly, Chakraborty et al. [16] proposed Reveal, which employed gated graph neural networks (GGNNs) to learn graph properties of code, and Li et al. [40] introduced IVDetect, which represented code as a program dependency graph (PDG) and used graph convolutional networks for vulnerability detection.

Building on these prior efforts, TaVer incorporates a code vulnerability detection component to capture latent temporal features that are critical for identifying potential vulnerabilities in code. Furthermore, while existing approaches have made significant progress in vulnerability detection, they often face substantial challenges in low-resource environments due to limited data and poor model generalization. To address this issue, TaVer introduces the Cross-Lingual Transfer Component that leverages cross-language feature sharing to enhance adaptability in low-resource settings. This approach reduces reliance on large-scale labeled data and significantly improves detection efficiency and accuracy, outperforming traditional baselines.

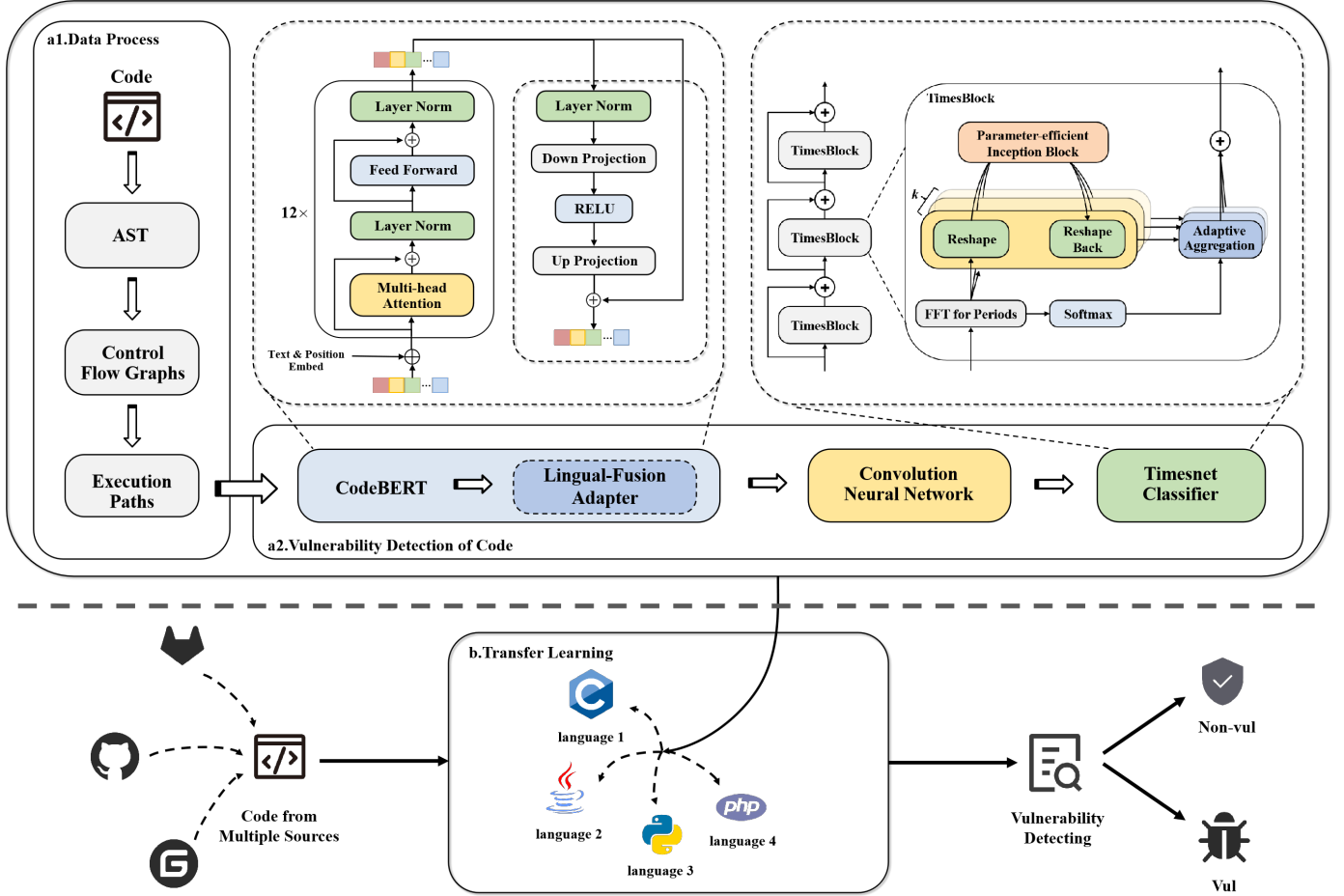


Fig. 2: The overall framework of TaVer.

3 THE TAVER MODEL

3.1 Overview

To address the challenges, we propose TaVer, a model designed to tackle the problem of low-resource code vulnerability detection, as illustrated in Fig. 2. TaVer focuses on learning common features from resource-rich programming languages and transferring this knowledge to low-resource languages. TaVer consists of two primary components: **a) Code Vulnerability Detection Component** and **b) Cross-Lingual Transfer Component**.

During training, the Code Vulnerability Detection Component first parses the code into an abstract syntax tree (AST) and generates a control flow graph (CFG). A greedy path selection algorithm is then used to identify representative execution paths, which are fed into a pre-trained model for feature extraction, allowing the model to capture internal attention and learn initial feature vectors. To further capture both short-term variations and long-term dependencies, the one-dimensional feature sequence is transformed into a two-dimensional tensor. By combining this transformation with a convolutional neural network (CNN) to extract inter-path attention, the model effectively learns complex temporal patterns, ultimately improving the accuracy of vulnerability detection.

The Cross-Lingual Transfer Component pre-trains model parameters on resource-rich programming languages and

transfers general knowledge for code vulnerability detection by fine-tuning downstream parameters for the target task in low-resource conditions. During pre-training, gradient descent updates the parameters of one or more source language models for vulnerability detection. After pre-training, the lingual-fusion transfer module is activated, applying the acquired knowledge to the target language, which has limited resources for vulnerability detection tasks.

The detailed algorithm of TaVer is presented in Algorithm 1. First, the parameters of model M are initialized (line 1). A dataset D_{s_i} from one or more source languages is selected for training model M , and its parameters are updated using gradient descent (lines 3-5). During the training process, the loss on the source language's test set is computed, and the corresponding gradient is used to update the parameters of M^* . Once the training on all source languages is completed, the pre-trained parameters M^* are obtained. Next, the parameters of model M_t are initialized (line 7), and the pre-trained parameters M^* are assigned to the corresponding parts of model M_t (line 8). These parameters are then frozen to prevent further updates (line 9). Finally, M_t is trained on the low-resource language dataset D_t , and the parameters A^* are updated through gradient descent (line 10). This ensures that the parameter adjustments align more effectively with the characteristics of the low-resource language while avoiding overfitting, ultimately enabling effective vulnerability detection for low-resource

programming languages.

3.2 Code Vulnerability Detection Component

Sections a1 and a2 in Fig. 2 illustrate the Code Vulnerability Detection Component, which includes: **a1) Data Preprocessing** and **a2) Code Vulnerability Detection**. During the data preprocessing phase, code snippets are decomposed into several representative execution paths. Each execution path represents a vulnerability-prone segment within the code snippet, highlighting areas where vulnerabilities may potentially emerge. In the code vulnerability detection phase, neural networks are used to encode each path individually, capturing both temporal information and complex patterns within the paths. This approach effectively identifies potential vulnerabilities, achieving efficient and accurate vulnerability detection.

3.2.1 Data Preprocessing

In the data preprocessing stage, code snippets are parsed to construct Abstract Syntax Trees (ASTs) and generate Control Flow Graphs (CFGs). A greedy algorithm is then applied to extract multiple representative execution paths from the CFG, selecting paths from the entry node to the exit node that cover key logic flows while filtering out irrelevant information. This approach ensures that critical paths associated with vulnerabilities are effectively identified, improving the overall detection accuracy.

3.2.2 Code Vulnerability Detection

In the code vulnerability detection phase, preprocessed code snippets are analyzed to predict the presence of vulnerabilities. This stage includes two key components: **execution path feature extraction** and **temporal pattern analysis**.

Execution Path Feature Extraction: In this stage, the target code snippet is represented by extracting key execution paths. To enhance the quality of code representation, TaVer capture both the features of individual paths and the relationships between different paths. CodeBERT is employed to encode each execution path extracted from the CFG, generating feature vectors for these paths. Pre-trained on large-scale code datasets, CodeBERT effectively captures both syntactic and semantic information from the code.

For longer code snippets, traditional models often truncate inputs due to length constraints, potentially omitting critical vulnerability-related information. Encoding each decomposed execution path individually ensures shorter path lengths, mitigating truncation risks and preserving vulnerability-relevant details. By leveraging CodeBERT for path vectorization, syntactic and semantic features are transformed into high-dimensional vectors, facilitating precise capture of vulnerability-specific details and enhancing detection accuracy.

The process is as follows: for each path P , all nodes (i.e., code statements) are sequentially arranged into a code segment S . Then, CodeBERT's tokenizer is applied to segment S , generating a sequence of tokens of length n . These token sequences are mapped into embedding vectors $\mathbf{X}^l = \{\mathbf{x}_1^l, \mathbf{x}_2^l, \dots, \mathbf{x}_n^l\}$ through an embedding layer. CodeBERT processes these embedding vectors using multiple transformer layers, learning the contextual information of

Algorithm 1: TaVer

Input: M, M_t : Code Vulnerability Detection Components

A: Cross-Lingual Transfer Component

D_s : n source language datasets

D_t : Target language dataset

Output: M_t : The trained Code Vulnerability Detection Component

- 1 Initialize the parameters of M
- 2 #Source Language Pre-Training
- 3 **for** $i = 1$ **to** n **do**
- 4 Use M to do gradient descent on M^* through D_{s_i} according to Eq 14
- 5 **end**
- 6 #Finetuning with the Cross-Lingual Transfer Component
- 7 Initialize the parameters of M_t
- 8 Assign M^* to the corresponding parts of the target model $M_t(M_t^* = M^* + A^*)$
- 9 Freeze the pre-trained parts of M_t (based on M^*)
- 10 Use M_t to do gradient descent on A^* through D_t according to Eq 14
- 11 **return** Component M_t

each token. The transformer layers utilize multi-head attention mechanisms, feed-forward networks, and layer normalization operations to progressively optimize the feature representation within each path.

$$\hat{\mathbf{X}} = \text{MultiHead}(\tilde{\mathbf{X}}, \tilde{\mathbf{X}}, \tilde{\mathbf{X}}) \quad (1)$$

$$\mathbf{X}^i = \text{LayerNorm}(\hat{\mathbf{X}} + \text{FFN}(\hat{\mathbf{X}})) \quad (2)$$

where $\mathbf{X}^i$ represents the output of the i_{th} transformer layer, $\text{MultiHead}(\cdot)$, $\text{FFN}(\cdot)$, and $\text{LayerNorm}(\cdot)$ denote the multi-head attention layer, feed-forward network, and layer normalization, respectively. After passing through l transformer layers, each path P is encoded into a set of context-aware embedding vectors $\mathbf{X}^l = \{\mathbf{x}_1^l, \mathbf{x}_2^l, \dots, \mathbf{x}_n^l\}$. Since a code snippet is decomposed into multiple paths, each path is independently encoded, and their feature representations are concatenated into a matrix $\mathbf{E} \in \mathbb{R}^{m \times d}$:

$$\mathbf{E} = \begin{bmatrix} \mathbf{e}_1 \\ \dots \\ \mathbf{e}_m \end{bmatrix} \quad (3)$$

where $\mathbf{e}_i$ represents the feature vector of the i_{th} path.

Next, by converting these features from a 1D sequence into 2D tensors, the model simultaneously captures both short-term and long-term temporal variations, enabling more precise identification of potential vulnerabilities within the code snippet.

Temporal Pattern Analysis: After initial feature extraction, the 1D path feature sequences are transformed into 2D tensors to capture complex temporal patterns within the paths. Vulnerability detection tasks require not only an analysis of code structure but also a precise understanding of the temporal characteristics that emerge during code execution.

Specifically, for the execution path feature sequences of length- T with C recorded variables, the original 1D structure is $\mathbf{X}_{1D} \in \mathbb{R}^{T \times C}$. A Fast Fourier Transform (FFT) is first performed to analyze the frequency distribution of these features, identifying the primary periodic characteristics within the execution path feature sequences:

$$\mathbf{A} = \text{Avg}(\text{Amp}(\text{FFT}(\mathbf{X}_{1D}))) \quad (4)$$

where $\text{FFT}(\cdot)$ and $\text{Amp}(\cdot)$ represent the FFT operation and the calculation of amplitude values, respectively. $\mathbf{A} \in \mathbb{R}^T$ represents the amplitude for each frequency, averaged over C dimensions. To remove high-frequency noise and focus on meaningful information, TaVer select only the top k frequencies with the largest amplitudes. These frequencies correspond to k period lengths $\{p_1, \dots, p_k\}$. Given the conjugate symmetry of the frequency domain, the focus is placed on frequencies within the range $\{1, \dots, \lfloor \frac{T}{2} \rfloor\}$.

$$f_1, \dots, f_k = \text{argTopk}(\mathbf{A}), \quad f_i \in \left\{1, \dots, \left\lfloor \frac{T}{2} \right\rfloor\right\} \quad (5)$$

$$p_i = \left\lfloor \frac{T}{f_i} \right\rfloor, \quad i \in \{1, \dots, k\} \quad (6)$$

where f_i denotes the i -th frequency, and p_i represents the period length associated with this frequency.

Based on the identified periodic characteristics, the 1D path feature sequences are reshaped into 2D tensors. The columns of each 2D tensor represent short-term temporal variations within a path, while the rows capture long-term temporal changes. Using the selected frequencies and their corresponding period lengths, the original 1D time series $\mathbf{X}_{1D} \in \mathbb{R}^{T \times C}$ is reshaped into multiple 2D tensors:

$$\mathbf{X}_{2D}^i = \text{Reshape}_{p_i, f_i}(\text{Padding}(\mathbf{X}_{1D})), i \in \{1, \dots, k\} \quad (7)$$

where $\text{Padding}(\cdot)$ performs zero-padding along the time dimension for compatibility with the $\text{Reshape}(\cdot)$ operation, and p_i and f_i represent the row and column dimensions of the transformed 2D tensor, respectively. $\mathbf{X}_{2D}^i \in \mathbb{R}^{p_i \times f_i \times C}$ represents the i -th time series reshaped based on frequency f_i , with rows and columns corresponding to periodic and intra-period changes. The result is a set of 2D tensors that display the temporal patterns of the execution paths across multiple periods. This transformation allows us to more effectively capture hidden vulnerability characteristics within the code by thoroughly analyzing both short-term and long-term temporal variations, improving detection accuracy.

To effectively capture both intra-period and inter-period variations, TimesBlock employs a residual structure that efficiently models temporal changes by using parameter-efficient initial blocks to handle these two distinct types of variations. Specifically, for a 1D input time series $\mathbf{X}_{1D} \in \mathbb{R}^{T \times C}$ of length- T , an embedding layer $\mathbf{X}_{1D}^0 = \text{Embed}(\mathbf{X}_{1D})$ is applied to map the raw input into deep feature representations, resulting in $\mathbf{X}_{1D}^0 \in \mathbb{R}^{T \times d_{\text{model}}}$.

At the l -th layer, the transformation process is defined as follows, with $\mathbf{X}_{1D}^{l-1} \in \mathbb{R}^{T \times d_{\text{model}}}$ serving as the input:

$$\mathbf{X}_{1D}^l = \text{TimesBlock}(\mathbf{X}_{1D}^{l-1}) + \mathbf{X}_{1D}^{l-1} \quad (8)$$

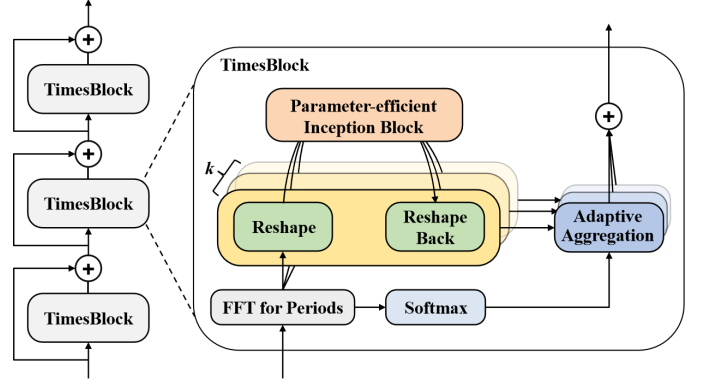


Fig. 3: The Structure of TimesBlock.

where $\mathbf{X}_{1D}^l$ represents the output of the l -th layer in the TimesBlock structure, and $\mathbf{X}_{1D}^{l-1}$ represents the input from the $(l-1)$ -th layer, serving as the residual connection for enhanced information flow.

In the TimesBlock layer, temporal variations across k different reshaped tensors are captured using highly efficient 2D convolutional kernels within the Inception block. These variations, derived from time-period transformations, are fused based on amplitude normalization (Fig. 3). The procedure is formalized as:

$$\mathbf{X}_{2D}^{l,i} = \text{Reshape}_{p_i, f_i}(\text{Padding}(\mathbf{X}_{1D}^{l-1})), i \in \{1, \dots, k\} \quad (9)$$

$$\hat{\mathbf{X}}_{2D}^{l,i} = \text{Inception}(\mathbf{X}_{2D}^{l,i}), i \in \{1, \dots, k\} \quad (10)$$

$$\hat{\mathbf{X}}_{1D}^{l,i} = \text{Trunc}(\text{Reshape}_{1, (p_i \times f_i)}(\hat{\mathbf{X}}_{2D}^{l,i})), i \in \{1, \dots, k\} \quad (11)$$

where $\mathbf{X}_{2D}^{l,i} \in \mathbb{R}^{p_i \times f_i \times d_{\text{model}}}$ denotes the l -th reshaped 2D tensor. The 2D convolutional kernels of the Inception module effectively extract multi-scale features, and the $\text{Trunc}(\cdot)$ function subsequently restores the processed features to their original length- T , preparing them for further aggregation. During this process, the Inception module aggregates both intra-period changes (columns) and inter-period variations (rows), thus enabling a comprehensive capture of temporal relationships across adjacent time points and periods. Inspired by Autoformer [45], amplitude values reflect the relative importance of each selected frequency and period. By adaptively aggregating multiple 1D representations, $\{\hat{\mathbf{X}}_{1D}^{l,1}, \dots, \hat{\mathbf{X}}_{1D}^{l,k}\}$, a weighted summation is performed based on the relative significance of each frequency and period. This generates an enriched feature representation for the input to the next layer:

$$\hat{\mathbf{A}}_{f_1}^{l-1}, \dots, \hat{\mathbf{A}}_{f_k}^{l-1} = \text{Softmax}(\mathbf{A}_{f_1}^{l-1}, \dots, \mathbf{A}_{f_k}^{l-1}) \quad (12)$$

$$\mathbf{X}_{1D}^l = \sum_{i=1}^k \hat{\mathbf{A}}_{f_i}^{l-1} \times \hat{\mathbf{X}}_{1D}^{l,i} \quad (13)$$

where $\hat{\mathbf{A}}_{f_i}^{l-1}$ represents the normalized importance of the i -th frequency, and $\hat{\mathbf{X}}_{1D}^{l,i}$ is the corresponding feature representation for the i -th frequency.

In this way, the TimesBlock module is capable of comprehensively capturing multi-scale temporal variations, providing a significantly more robust feature learning mechanism compared to directly modeling 1D time series. This results in improved accuracy for code vulnerability detection.

3.3 Cross-Lingual Transfer Component

The goal of transfer learning is to leverage prior knowledge gained from training on one or more source languages and quickly transfer it to low-resource target languages, enabling efficient code vulnerability detection. To achieve this, the Cross-Lingual Transfer Component employs meta-learning and second-order gradient techniques, facilitating few-shot cross-lingual learning. As illustrated in Fig. 2, this component operates in two phases: **b1) the pre-training phase** and **b2) the lingual-fusion fine-tuning phase**. During the pre-training phase, the model extracts universal features of code vulnerabilities from multilingual datasets, learning patterns that are applicable across different programming languages. In the lingual-fusion fine-tuning phase, the model efficiently fine-tunes downstream parameters for the target language based on the pretrained model, addressing the specific needs of the target language. This approach ensures effective knowledge transfer, allowing the model to quickly adapt in low-resource scenarios while maintaining high performance in code vulnerability detection tasks.

3.3.1 Pre-training for Cross-Lingual Vulnerability Detection

During the pre-training phase, we establish N training tasks, each based on a high-resource programming language. Each task involves independent training and testing phases. For a given training task m , we assign a Code Vulnerability Detection Component f_{θ}^m with model parameters θ_m , initialized as ϕ^0 .

For the large-sample language dataset S_m , the parameters θ_m are updated using gradient descent, with the update rule for this task expressed as:

$$\theta'_m \leftarrow \theta_m - \eta \nabla_{\theta_m} l^m(f_{\theta}^m) \quad (14)$$

where η represents the learning rate, and $l^m(f_{\theta}^m)$ is the loss function that quantifies the discrepancy between the predicted classifications and the true vulnerability status of the code snippets. Through this process, TaVer continuously updates the parameters of f_{θ}^m , transitioning the model parameters from ϕ^0 to ϕ^1 . After completing training on task m , we move on to another training task n and repeat this procedure, further updating the parameters from ϕ^1 to ϕ^2 . When all training tasks are complete, we obtain the initial parameters ϕ of the Code Vulnerability Detection Component, which are then used for low-resource target languages.

As outlined above, during the pre-training phase, TaVer progressively learns generalizable knowledge of code vulnerability detection from high-resource source languages. This accumulated knowledge forms a foundation of universal features, which are then fine-tuned in the subsequent phase.

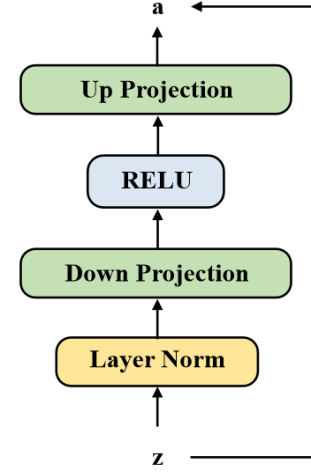


Fig. 4: Lingual-Fusion module.

3.3.2 Lingual-Fusion Finetuning

TaVer leverages the Cross-Lingual Transfer Component to apply the knowledge accumulated during pretraining to code vulnerability detection in the target language. Specifically, building on the pretrained network parameters ϕ , TaVer initializes a new Cross-Lingual Transfer Component, which is integrated into the Transformer architecture of CodeBERT and applied after the final encoder layer to enhance cross-lingual adaptability.

During this phase, the core parameters of the pretrained model remain frozen, and only the parameters associated with the Cross-Lingual Transfer Component are updated, ensuring parameter-efficient adaptation. Specifically, the extracted key code execution paths are encoded by the encoder and processed within the Cross-Lingual Transfer Component, which balances general-purpose knowledge with target language-specific characteristics while preserving task-critical information. Features are compacted to emphasize relevant aspects for vulnerability detection, undergo a non-linear transformation to capture complex patterns, and are then expanded back to their original dimensionality. The transformed features are merged with the original encoder outputs, preserving shared knowledge while incorporating task-specific adjustments. This balanced representation enables effective cross-lingual code vulnerability detection, allowing the model to generalize across diverse programming languages.

Additionally, it is worth noting that the Cross-Lingual Transfer Component is deliberately applied after the final encoder layer rather than after each layer. This design choice is based on the observation that task-specific knowledge tends to be concentrated in the later layers of neural networks, while earlier layers primarily learn general features. This observation is supported by Yosinski et al. [46] (NIPS 2014), who showed that earlier layers in transfer learning capture general features, while the later layers focus on learning task-specific ones.

The architecture of the Cross-Lingual Transfer Component is shown in Fig. 4. It includes layer normalization, a down-projection layer, a non-linear activation layer (ReLU), and an up-projection layer with residual connections. The

corresponding formula is as follows

$$\mathbf{a}^l = \text{LFusion}(\mathbf{z}^l) = \mathbf{z}^l + \mathbf{W}_u^l \text{ReLU}(\mathbf{W}_d^l (\text{LN}(\mathbf{z}^l))) \quad (15)$$

where $\text{LFusion}(\cdot)$ represents the process of the Lingual-Fusion module, $\mathbf{z}^l$ is the output of the final encoder layer, LN denotes layer normalization, $\mathbf{W}_u$ and $\mathbf{W}_d$ represent the weights of the up-projection and down-projection layers, respectively.

This structure allows for minimal adjustment of the encoder's final layer, ensuring that the Cross-Lingual Transfer Component effectively enhances the model's adaptability to the target language. As a result, it enables efficient code vulnerability detection in low-resource programming languages.

4 EXPERIMENT SETUP

4.1 Datasets and Preprocessing

To evaluate the effectiveness of TaVer, we utilized two datasets: the D2A dataset [27] and the CrossVul dataset [28]. The D2A dataset comprises C language code at the function level, while the CrossVul dataset is constructed at the file level and includes vulnerability detection data for three programming languages: Java, Python, and PHP. Compared to D2A, the CrossVul dataset contains fewer samples. Table 1 provides detailed statistics for both datasets, where the 3rd and 4th columns show the number of vulnerable and non-vulnerable samples, the 5th column displays the proportion of vulnerable samples, and the 6th column provides the total number of samples for each dataset.

The D2A dataset, focusing on function-level C language code, includes common vulnerability types such as Buffer Overflow, Null Pointer Dereference, Resource Leaks, Use-After-Free, and Arithmetic Errors, reflecting real-world vulnerabilities that are often rare. The CrossVul dataset, constructed at the file level, includes data for Java, Python, and PHP, covering vulnerabilities aligned with key CWE categories, including Improper Input Validation (CWE-20), Command Injection (CWE-77), Cross-Site Scripting (CWE-79), SQL Injection (CWE-89), and File Inclusion (CWE-98). Each language subset in the CrossVul dataset is balanced, containing an equal number of vulnerable and non-vulnerable samples, which facilitates controlled experiments for cross-lingual transfer learning.

In our experiments, we designated portions of these datasets as the source domain, with the remaining portions serving as the target domain. The resource-rich D2A dataset was consistently used as the source domain, while the Java, Python, and PHP subsets of the CrossVul dataset were used as target domains. This setup simulates a cross-lingual transfer learning scenario where knowledge from a high-resource language (C language) is transferred to low-resource languages.

For evaluation, all datasets were randomly and uniformly split into training (80%), validation (10%), and testing (10%) sets. During the pretraining stage, the training set of the source domain (D2A) was used to pretrain the model, while its validation set was employed to tune parameters, and its test set was used to assess the initial performance

of the pretrained model. During the fine-tuning stage, the target domain datasets (CrossVul) were similarly divided. The training set of the target domain was used to adapt the model to the target language, the validation set was used to refine the model parameters, and the test set was used for the final evaluation of the cross-lingual transfer learning performance.

4.2 Evaluation Metrics

Following previous studies [14], [29], [40], we employed four widely used classification evaluation metrics: **Accuracy**, **Precision**, **Recall** and **F1-Score**.

Accuracy: Accuracy measures the proportion of correctly predicted samples out of the total number of samples, serving as an indicator of the model's overall correctness. It is defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (16)$$

Precision: Precision evaluates the proportion of correctly predicted vulnerable code fragments among those predicted to be vulnerable, reflecting the model's ability to accurately identify positive cases. It is defined as:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (17)$$

Recall: Recall measures the proportion of truly vulnerable code fragments that the model successfully detects. It is defined as:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (18)$$

F1-Score: F1-Score is the harmonic mean of Precision and Recall, providing a comprehensive assessment of the model's performance in handling positive samples. It is defined as:

$$\text{F1-Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (19)$$

In these formulas, the following symbols represent different types of classification results:

- **TP:** The number of vulnerable code fragments correctly identified as vulnerable (True Positives).
- **TN:** The number of non-vulnerable code fragments correctly identified as non-vulnerable (True Negatives).
- **FP:** The number of non-vulnerable code fragments incorrectly classified as vulnerable (False Positives).
- **FN:** The number of vulnerable code fragments incorrectly classified as non-vulnerable (False Negatives).

4.3 Baselines

To evaluate the effectiveness of our approach, we compared it against five state-of-the-art techniques: CodeBERT [29], ReGVD [30], SimAdapter [31], and EPVD [26].

- **CodeBERT** is a pre-trained programming language model based on BERT, supporting six programming languages: Java, Python, JavaScript, PHP, Ruby, and Go. It utilizes masked language modeling (Devlin et al., 2019) and replaced token detection objectives (Clark

TABLE 1: The Statistics of Datasets

Dataset	Language	Vul	Non-vul	Vulnerable Ratio (%)	Total
D2A	C	2793	3059	47.73%	5852
CrossVul	Java	525	603	46.54%	1128
CrossVul	Python	513	565	47.59%	1078
CrossVul	PHP	879	930	48.59%	1809

et al., 2020). CodeBERT has been applied directly to predict vulnerabilities in code snippets and has demonstrated strong performance across various software engineering tasks (Feng et al., 2020).

- **ReGVD** is built on residual graph neural networks and is trained by capturing token relationships through a sliding window mechanism. This model leverages graph-based representations of code to enhance vulnerability detection.
- **Simadapter** is a cross-lingual model that employs adapters and attention mechanisms to learn similarities between source and target languages. It improves cross-lingual adaptability by integrating features from multiple languages and introducing a fusion-guided loss function to balance adapter weights between the source and target languages.
- **EPVD** is a path-based vulnerability detection model that decomposes control flow graphs (CFGs) into multiple execution paths for feature extraction. It encodes these paths using a pre-trained model and integrates them through a convolutional neural network (CNN) for vulnerability detection tasks.

4.4 Training Details

This section provides the hyperparameters used during the training of TaVer. The block size for token embeddings was set to 400, with a filter size of 3 and a hidden dimension of 128 for convolutional operations. A single convolutional kernel and one processing layer ($e_layers = 1$) were utilized to efficiently extract temporal patterns. Training was performed with a batch size of 5, while evaluation used a batch size of 8. The AdamW optimizer was employed with a learning rate of $2e-5$ and a maximum gradient norm of 1.0 to ensure stable updates. All experiments were implemented using the PyTorch framework with Python 3.8 and executed on a machine equipped with an Intel Core i9-13900K CPU, 128 GB of RAM, and an NVIDIA RTX A6000 GPU with 46 GB of VRAM.

5 EXPERIMENTAL RESULTS

This section presents six experiments designed to evaluate the effectiveness of TaVer.

RQ1. Comparisons to Baselines

RQ2. Ablation Study on Temporal Modeling for Code Vulnerability Detection

RQ3 Impact of Varying the Number of Kernels in Convolutional Blocks on Performance with a Fixed Number of Temporal Layers

RQ4. Ablation Study on Cross-Lingual Transfer Learning for Code Vulnerability Detection

RQ5. Different Language Combinations in Transfer Learning for Code Vulnerability Detection

RQ6. Impact of Fine-Tuning at Different Positions in the Network on Cross-Lingual Code Vulnerability Detection

RQ1 evaluates TaVer's overall performance in low-resource code vulnerability detection. RQ2 aims to assess the effectiveness of the Code Vulnerability Detection Component, while RQ3 investigates the role of convolutional kernels in extracting complex temporal patterns, both aiming to better capture temporal relationships in code execution paths. RQ4 assesses the effectiveness of the Cross-Lingual Transfer Component, RQ5 investigates the impact of different language combinations, and RQ6 analyzes fine-tuning at different network layers, all contributing to addressing the challenges of data scarcity and enhancing cross-language knowledge transfer.

5.1 Answer to RQ1: Comparisons to Baselines

Motivation. In this section, we evaluate the performance of TaVer in low-resource code vulnerability detection tasks, comparing it against state-of-the-art approaches to validate its effectiveness.

Methodology. We compared TaVer with four recent baseline models—CodeBERT, ReGVD, SimAdapter, and EPVD—focusing on Java, Python, and PHP, which are considered low-resource languages relative to C. Both TaVer and SimAdapter employ transfer learning strategies, using the resource-rich C language dataset as the source language and transferring knowledge to the target language datasets of Java, Python, and PHP. In contrast, the other baseline models were trained directly on these target datasets. We comprehensively evaluated each model's performance using four evaluation metrics: Accuracy, Precision, Recall, and F1-Score.

Results. Table 2 presents the experimental results of TaVer compared to baseline models across different target language datasets. The experimental results show that the performance of TaVer outperforms the baseline approaches in code vulnerability detection in terms of Accuracy, Precision, Recall, and F1-Score, e.g., compared with baseline approaches, TaVer improved Accuracy by 4.63%, 124.82%, 17.71%, and 25.44%, and Precision by 30.59%, 120.18%, 63.05%, and 43.41%, respectively. Notably, on the PHP dataset, TaVer demonstrated significant enhancements, achieving an Accuracy of 54.95%, a Precision of 43.59%, a Recall of 47.22%, and an F1-Score of 45.33%. In comparison, CodeBERT and SimAdapter recorded Accuracy values of 44.51% and 40.66%, respectively, while EPVD performed the worst, with an Accuracy of only 28.57%, highlighting its limitations in processing PHP code. In particular, TaVer achieved an average improvement of approximately 30 percentage points in Precision, Recall, and F1-Score compared to the best baseline, along with a 15 percentage point increase in Accuracy. These findings demonstrate TaVer's

TABLE 2: Effectiveness of TAVer and Baseline Models in Code Vulnerability Detection on Three Low-Resource Programming Languages (%)

Approaches	Java				Python				PHP				Average			
	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
CodeBERT	51.75	34.29	27.27	30.38	42.20	31.03	17.31	22.22	44.51	30.67	31.94	31.29	46.15	32.00	25.51	27.96
ReGVD	50.88	25.00	13.64	17.65	36.70	22.58	13.46	16.87	47.25	29.31	23.61	26.15	23.53	18.98	26.38	21.91
Simadapter	48.25	33.33	34.09	33.71	37.61	27.78	19.23	22.73	40.66	26.32	27.78	27.03	44.94	25.63	16.90	20.22
EPVD	23.68	19.72	31.82	24.35	18.35	15.09	15.38	15.24	28.57	22.12	31.94	26.14	42.17	29.14	27.03	27.82
TaVer	57.89	44.74	38.64	41.46	45.87	37.04	19.23	25.32	54.95	43.59	47.22	45.33	52.90	41.79	35.03	37.37

stronger performance in code vulnerability detection tasks for target languages, further validating the advantages of transfer learning and temporal modeling in this domain.

Both CodeBERT and ReGVD perform well in capturing local information in the code, but each has limitations. CodeBERT primarily detects vulnerabilities by capturing syntactic and semantic information, while ReGVD relies on sliding windows to capture local dependencies between tokens. However, neither approach explicitly models the structural relationships in code (e.g., control flow and execution paths), which are crucial for capturing the global context needed to detect complex vulnerabilities. In contrast, TaVer combines temporal modeling with cross-lingual transfer learning, enabling it to capture both sequential and structural features of code, leading to a deeper understanding of the execution process and significantly improving detection accuracy, especially in low-resource languages.

The primary distinction between SimAdapter and TaVer lies in their implementation. SimAdapter introduces a dedicated SimAdapter layer that leverages an attention mechanism to learn similarities between the source and target languages, facilitating knowledge transfer. However, languages such as C, Java, Python, and PHP exhibit significant differences in programming paradigms, syntax rules, and application scenarios. These weak similarities can introduce noise during the transfer process, leading to degraded performance. In contrast, TaVer avoids relying on such similarity-based assumptions, demonstrating more robust and consistent performance in cross-lingual vulnerability detection tasks.

EPVD relies on control flow graphs to decompose execution paths, but it struggles to capture dynamic changes in the code execution process due to its static nature. In contrast, TaVer enhances temporal feature extraction, enabling a more comprehensive processing of execution sequences in the code, thereby addressing EPVD's limitations. These design advantages explain why TaVer consistently surpasses the baseline models across multiple evaluation metrics, particularly in complex transfer learning tasks, further validating the effectiveness of combining temporal modeling with transfer learning.

Conclusion. TaVer has demonstrated its effectiveness in detecting code vulnerabilities in low-resource settings, significantly outperforming the baseline models.

5.2 Answer to RQ2: Ablation Study on Temporal Modeling for Code Vulnerability Detection

Motivation. With the increasing complexity of software systems, the implicit temporal relationships between code fragments (e.g., the calling sequences among multiple code

modules) play a crucial role in code vulnerability detection. Accurately capturing these implicit temporal dynamics is essential for comprehensively understanding potential risks during code execution. Therefore, we compare the first component of our model, the Code Vulnerability Detection Component, with several state-of-the-art temporal modeling approaches to validate its effectiveness.

Methodology. In this experiment, we integrate three state-of-the-art temporal modeling approaches—ModernTCN [47], RWKV-TS [48], and FPT [49]—into our model as replacements for the first component, the Code Vulnerability Detection Component, while keeping the second component, the Cross-Lingual Transfer Component, unchanged.

- **ModernTCN:** This model modernizes the traditional Temporal Convolutional Network (TCN) with larger effective receptive fields (ERFs), enabling it to better capture both short-term and long-term temporal dependencies. It also integrates architectural designs inspired by transformers while maintaining the efficiency of convolution-based models.
- **RWKV-TS:** An RNN-inspired model with linear time and space complexity, RWKV-TS emphasizes efficient long-term dependency modeling through token shifts and a WKV (weighted key-value) operator, mimicking attention mechanisms.
- **FPT:** A frozen pre-trained transformer adapted from natural language processing for time series analysis, leveraging self-attention to generalize across tasks and capture intricate patterns in sequential data.

We also included a simple classifier, MLP, for comparison, representing an approach that does not leverage the temporal features between code fragments. In this approach, the operation `seq_embeds = seq_embeds[:, 0, :]` is used to extract only the features from the first time step of the entire sequence. The first time step from the CodeBERT encoder typically contains global information, providing a concise summary of the code fragment. The global features are then input into a CNN for local feature extraction, followed by concatenation with the global features through a linear layer. The classification is completed using the MLP. This approach has the advantage of lower computational complexity, as it reduces the amount of temporal information processed. However, by using only the features from the first time step, it overlooks the temporal dynamics of the entire sequence, particularly the complex temporal patterns in execution paths, which poses a significant limitation.

Results. Comparative experiments were conducted on three target language datasets, with the C language dataset used as the source language. As shown in Table 3, the experimental results indicate that the TaVer approach out-

TABLE 3: Effectiveness of Temporal Modeling Approaches for Code Vulnerability Detection (%)

Approaches	Java				Python				PHP				Average			
	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
ModernTCN	58.77	41.18	15.91	22.95	44.04	23.53	7.69	11.60	47.80	29.82	23.61	26.36	50.20	31.51	15.74	20.30
RWKV-TS	57.89	44.44	36.36	40.00	41.28	28.57	15.38	20.00	45.05	36.00	50.00	41.86	48.07	36.34	33.91	33.95
FPT	47.37	34.00	38.64	36.17	40.37	29.03	17.31	21.69	40.66	32.00	44.44	37.21	42.8	31.68	33.46	31.69
None(MLP)	56.14	35.00	15.91	21.88	44.95	27.78	9.62	14.28	53.30	38.98	31.94	35.11	51.46	33.92	19.16	23.76
TaVer	57.89	44.74	38.64	41.46	45.87	37.04	19.23	25.32	54.95	43.59	47.22	45.33	52.90	41.79	35.03	37.37

TABLE 4: Impact of Convolutional Kernel Numbers on Code Vulnerability Detection Performance(%)

e_layers:num_kernels	Accuracy	Precision	Recall	F1
1 : 3	53.51	37.84	31.82	34.57
1 : 2	57.02	41.94	29.54	34.67
1 : 1	57.89	44.74	38.64	41.46

performs the other four approaches in terms of Accuracy, Precision, Recall, and F1-score. Specifically, compared to other temporal modeling approaches, TaVer achieves an average Accuracy improvement of 5.38%, 10.05%, and 23.6%, respectively. Regarding average Precision, TaVer exceeds the other temporal modeling approaches by 32.62%, 15%, and 31.91%, respectively, and outperforms None (MLP) by 23.2%. In terms of average Recall, TaVer improves by 122.55% over ModernTCN, demonstrating its exceptional capability in comprehensively identifying potential vulnerabilities. Moreover, for average F1-score, TaVer outperforms RWKV-TS and FPT by 10.07% and 17.92%, respectively. These results clearly demonstrate that the first component of the TaVer approach, the Code Vulnerability Detection Component, can significantly enhance the overall performance of code vulnerability detection by effectively capturing the temporal features within code fragments.

ModernTCN achieves the highest Accuracy on Java (58.77%) and performs reasonably well on PHP (47.80%). However, its performance on Python is significantly lower, with a Recall of only 7.69% and an F1-score of 11.60%. On average, ModernTCN exhibits the weakest Recall (15.74%) and F1-score (20.3%) among all the approaches. This discrepancy arises because ModernTCN prioritizes short-term temporal dependencies, leveraging large effective receptive fields (ERFs) for local feature capture. While this makes it efficient for simpler sequences like Java, it struggles to capture the intricate long-term relationships present in datasets like Python, resulting in weak Recall and F1-scores.

RWKV-TS demonstrates a strong balance across all datasets, with a particularly high Recall on PHP (50%) and Precision on Java (44.44%). Its average F1-score (33.95%) is a significant improvement over ModernTCN and highlights its consistency across different languages. This performance can be attributed to its efficient token-shift mechanism and WKV (weighted key-value) operator, which emulate attention mechanisms to capture both short-term and long-term dependencies effectively. However, while RWKV-TS performs well, it is slightly outperformed by TaVer in Precision and F1-score, indicating that its simpler temporal mixing approaches might not fully capture the nuanced patterns required for code vulnerability detection.

FPT exhibits moderate performance, with strong Recall on Java (38.64%) and a balanced performance across other

metrics. However, its overall Accuracy (42.8% average) is lower than other temporal modeling approaches. This result is likely due to its frozen pre-trained architecture, which leverages global sequence representations but lacks task-specific fine-tuning for code vulnerability detection. The model struggles to generalize well across all datasets, particularly for complex temporal dynamics. Nonetheless, FPT's ability to maintain competitive Recall and F1-scores indicates that pre-trained transformers are effective at capturing general temporal patterns, albeit with some limitations in domain-specific adaptation.

The MLP classifier demonstrates significant limitations in capturing temporal dependencies, as reflected in its low Recall (19.16% average), which is almost the weakest among all models. This shortcoming highlights its inability to effectively identify a substantial portion of true vulnerabilities, especially in scenarios requiring temporal analysis. By relying solely on the global feature vector from CodeBERT, the MLP approach simplifies computation but sacrifices the ability to detect code vulnerabilities when there are complex calling relationships between multiple code fragments. While it achieves relatively high Precision (38.98% on PHP), this comes at the cost of missing many true vulnerabilities, making it less reliable for comprehensive code vulnerability detection. These results underscore the critical importance of leveraging temporal modeling for more effective detection.

Conclusion. The Code Vulnerability Detection Component consistently outperformed state-of-the-art temporal modeling approaches across all four metrics. This highlights its capability to capture both short-term and long-term temporal dynamics, significantly enhancing code vulnerability detection. By effectively modeling the implicit temporal relationships in code fragments, TaVer significantly improves the performance of code vulnerability detection.

5.3 Answer to RQ3: Impact of Varying the Number of Kernels in Convolutional Blocks on Performance with a Fixed Number of Temporal Layers

Motivation. In this section, we investigate how varying the number of convolutional kernels impacts the performance of code vulnerability detection when the number of temporal layers is fixed at 1. Convolutional kernels are responsible for extracting local features, and changes in

TABLE 5: Performance Comparison of Cross-Lingual Transfer Learning Approaches for Code Vulnerability Detection (%)

Approaches	Java				Python				PHP				Average			
	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
<i>ChildPruning</i>	28.95	20.63	29.55	24.3	18.35	10.64	9.62	10.10	27.47	23.21	36.11	28.26	24.92	18.16	25.09	20.89
<i>DiffPruning</i>	32.46	22.95	31.82	26.67	21.10	14.58	13.46	14.00	26.37	20.75	30.56	24.72	26.64	19.43	25.28	21.80
<i>SAM</i>	31.58	20.69	27.27	23.53	20.18	14.29	13.46	13.86	29.67	24.07	36.11	28.89	27.14	19.68	25.61	22.09
<i>FullTuning</i>	34.21	24.59	34.09	28.57	21.10	11.36	9.62	10.42	29.67	24.07	36.11	28.89	28.33	20.01	26.61	22.63
<i>TaVer</i>	57.89	44.74	38.64	41.46	45.87	37.04	19.23	25.32	54.95	43.59	47.22	45.33	52.90	41.79	35.03	37.37

the number of kernels directly influence how the model processes input features. The objective is to understand how different kernel quantities affect feature extraction efficiency and overall model performance.

Methodology. We fixed the number of temporal layers (e_layers) at 1 while varying the number of convolutional kernels ($num_kernels$). These kernels are a critical component of the TimesBlock, responsible for extracting both short-term and long-term local features from different periods of the time series. We tested configurations with 1, 2, and 3 kernels and evaluated the model's performance on the vulnerability detection task using multiple metrics, including Accuracy, Precision, Recall and F1-Score.

Results. As shown in Table 4, the experimental results indicate that the model achieves the best performance in terms of Accuracy, Precision, Recall, and F1-Score when the number of convolutional kernels is set to 1, reaching 57.89%, 44.74%, 38.64%, and 41.46%, respectively. However, as the number of kernels increases, the overall model performance gradually declines. Specifically, when the number of kernels is set to 2 or 3, the F1-Score drops to around 35%, indicating a significant reduction in the model's generalization ability. Although convolutional kernels can extract features across different time steps, an excessive number of kernels may introduce redundant features, thereby weakening the model's effectiveness. With a fixed number of temporal layers, increasing the number of kernels adds complexity to the model, which in turn reduces its generalization ability. Therefore, selecting an appropriate number of convolutional kernels is crucial for maintaining model efficiency and generalization capacity.

Conclusion. The TaVer model performs optimally with 1 convolutional kernel, while increasing the number of kernels leads to degraded performance. Optimizing the number of convolutional kernels can significantly enhance model performance in code vulnerability detection tasks, and a simpler model configuration with fewer kernels offers better generalization ability.

5.4 Answer to RQ4: Ablation Study on Cross-Lingual Transfer Learning for Code Vulnerability Detection

Motivation. This experiment aims to compare the effectiveness of different transfer learning strategies in low-resource code vulnerability detection tasks. We evaluate the second component of the TaVer model, the Cross-Lingual Transfer Component, against several state-of-the-art transfer learning approaches to validate its efficiency and robustness in such scenarios.

Methodology. We integrate four advanced transfer learning strategies—ChildPruning [50], DiffPruning [51], SAM [52] and FullTuning—into our model as replacements

for the second component (Cross-Lingual Transfer Component), while keeping the first component (Code Vulnerability Detection Component) unchanged.

- **ChildPruning:** This approach updates only a subset of parameters, known as the "child network," by masking gradients of the remaining parameters during back-propagation. It strategically identifies task-relevant parameters, preventing overfitting and maintaining generalization, especially in low-resource settings.
- **DiffPruning:** Introduces a sparse, task-specific "diff vector" added to the pre-trained parameters. This vector is optimized with a differentiable L0-norm penalty, enabling parameter-efficient adaptation.
- **SAM(Second-order Approximation Method):** Utilizes second-order optimization to selectively fine-tune a small subset of parameters. By approximating the Hessian matrix, SAM achieves parameter efficiency while maintaining stability.
- **FullTuning:** A baseline approach where all parameters of the model are fine-tuned. This approach allows complete adaptation but is highly parameter inefficient.

Results. The C language dataset was used as the source language, while Java, Python, and PHP datasets were used as target languages. Four evaluation metrics—Accuracy, Precision, Recall, and F1-score—were employed to assess the models. Table 5 compares the performance of these transfer learning approaches across various target languages. The experimental results demonstrate that TaVer outperforms all other transfer learning approaches across all target languages. Specifically, TaVer achieves significant improvements in average Recall, outperforming ChildPruning, DiffPruning, SAM and FullTuning by 39.62%, 38.57%, 36.78%, and 31.64%, respectively. In terms of average F1-score, TaVer also shows substantial advantages, with increases of 78.89%, 71.42%, 69.17%, and 65.13% over these approaches. Additionally, TaVer consistently leads in both average Accuracy and Precision compared to the other approaches.

These results highlight the effectiveness of the second component of the TaVer approach, the Cross-Lingual Transfer Component, in efficiently leveraging knowledge from resource-rich languages and transferring it to low-resource tasks. Specifically, TaVer fine-tunes parameters efficiently in the later layers to better capture task-specific features while relying on earlier layers to extract general features. This design enables effective cross-lingual knowledge transfer while maintaining parameter efficiency. Compared to existing transfer learning approaches, TaVer achieves a better balance between cross-lingual generalization and task-specific adaptation, delivering superior performance in code vulnerability detection under low-resource conditions.

ChildPruning performs the weakest overall, with the lowest average Accuracy (24.92%), Precision (18.16%), and F1-Score (20.89%). Its Recall (25.09%) is lower than both DiffPruning (25.28%) and SAM (25.61%), further highlighting its limitations. The selective parameter updates of ChildPruning restrict its ability to adapt to the complexity of cross-lingual tasks, limiting the model's generalization ability and resulting in consistently poor performance across all metrics.

DiffPruning performs moderately, achieving an average Accuracy of 26.64%, Precision of 19.43%, Recall of 25.28%, and F1-Score of 21.8%. Its sparse diff vector allows the model to retain pre-trained knowledge while adapting to the target task. However, the sparsity of the updates restricts the model's flexibility to fully capture task-specific patterns, leading to only moderate improvements over ChildPruning.

SAM achieves slightly better results than DiffPruning and ChildPruning, with an average Accuracy of 27.14% and Recall of 25.61%. However, its reliance on second-order approximations for parameter selection limits its adaptability to complex cross-lingual tasks. This approach balances parameter efficiency and stability but may underperform in capturing the nuanced dynamics of highly divergent datasets, constraining its overall performance.

FullTuning fully updates all parameters, achieving relatively high Recall compared to parameter-efficient approaches like ChildPruning and DiffPruning. However, this approach comes at the cost of reduced precision and increased susceptibility to overfitting in low-resource settings, making it less effective overall and still inferior to TaVer across all metrics.

Conclusion. The Cross-Lingual Transfer Component of TaVer surpasses state-of-the-art transfer learning approaches by effectively balancing cross-lingual generalization and task-specific adaptation. Through efficient fine-tuning, TaVer achieves outstanding performance across all evaluation metrics.

5.5 Answer to RQ5: Different Language Combinations in Transfer Learning for Code Vulnerability Detection

Motivation. This section explores the impact of different language combinations on transfer learning effectiveness in code vulnerability detection. With Java, Python, and PHP datasets having relatively similar resources, the experiment shifts focus from high-to-low resource knowledge transfer to examining how language similarities or differences influence outcomes, emphasizing the critical role of source-target language similarity in cross-lingual knowledge transfer.

Methodology. The experiment sets up three different transfer learning combinations involving Java, Python, and PHP datasets. These strategies include both single-source and multi-source transfer learning, aimed at studying how different language combinations affect overall performance. The specific transfer combinations are as follows:

- Java → PHP
- Python → PHP
- Java + Python → PHP

For each combination, we pre-trained the model on the source language dataset and fine-tuned it on the target language dataset using the Cross-Lingual Transfer Component.

TABLE 6: Performance Comparison of Different Language Combinations in Transfer Learning (%)

Approach	Precision	Recall	F1
CodeBERT _(PHP)	30.67	31.94	31.29
TaVer _(Java-PHP)	24.24	11.11	15.24
TaVer _(Python-PHP)	31.88	30.56	31.21
TaVer _(Java + Python-PHP)	33.00	45.83	38.37

The model's performance was evaluated using three key metrics: Precision, Recall, and F1-Score, with results compared to the CodeBERT model, which performed relatively well among the baseline models without transfer learning. The focus was on observing the effect of transfer learning between different language combinations and determining whether multi-source transfer outperforms single-source transfer.

Results. As shown in Table 6, the experimental results for the Java → PHP transfer scenario indicate that TaVer performs slightly lower than the baseline CodeBERT in terms of Precision, Recall, and F1-Score, suggesting weaker overall performance. This implies that significant differences in syntax and structure between Java and PHP hinder effective knowledge transfer. Specifically, Java is a strongly-typed, object-oriented language, while PHP is a loosely-typed, scripting language. These fundamental differences in type systems and runtime environments make effective feature transfer from the source language to the target language challenging. Additionally, notable differences between Java and PHP in error handling mechanisms and memory management further complicate cross-language transfer, weakening overall transfer learning performance.

In contrast, the Python → PHP combination shows improvement in Precision, although Recall and F1-Score are slightly lower than the CodeBERT baseline. Overall, compared to the Java → PHP transfer scenario, the Python → PHP combination is relatively more balanced and effective.

For the Java + Python → PHP multi-source transfer strategy, results show improvements across all metrics compared to single-source transfer. Precision reached 33%, Recall significantly increased to 45.83%, and F1-Score rose to 38.37%. These experimental results indicate that leveraging multiple source languages significantly enhances the model's generalization ability and improves the overall effectiveness of transfer learning. This demonstrates that combining knowledge from multiple source languages helps better capture features in the target language, thereby enhancing the accuracy and robustness of code vulnerability detection.

Conclusion. The results highlight that language similarity plays a critical role in the effectiveness of transfer learning. Multi-source transfer strategies outperform single-source transfers, particularly when the languages involved share similar syntactic structures. This demonstrates that carefully selecting source languages and employing multi-source transfer strategies can significantly improve the generalization capability of models in code vulnerability detection.

TABLE 7: Effect of Fine-Tuning at Different Network Positions on Cross-Lingual Code Vulnerability Detection (%)

Approaches	Java				Python				PHP				Average			
	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1	Accuracy	Precision	Recall	F1
TaVer-PerLayer	33.33	20.37	25.00	22.45	22.94	13.64	11.54	12.50	30.77	25.00	37.50	30.00	29.01	19.67	24.68	21.65
TaVer	57.89	44.74	38.64	41.46	45.87	37.04	19.23	25.32	54.95	43.59	47.22	45.33	52.90	41.79	35.03	37.37

5.6 Answer to RQ6: Impact of Fine-Tuning at Different Positions in the Network on Cross-Lingual Code Vulnerability Detection

Motivation. This experiment investigates the effect of fine-tuning at different positions in the network on cross-lingual code vulnerability detection. Specifically, it compares fine-tuning applied exclusively in the final encoder layer (as in TaVer) with fine-tuning applied across all encoder layers (as in TaVer-PerLayer). The hypothesis is that task-specific knowledge is primarily concentrated in the later layers of neural networks, while earlier layers capture more general features. By restricting fine-tuning to the final layer, the model can adapt to task-specific features while preserving the general representations learned in earlier layers.

Methodology. To validate this hypothesis, we evaluate two approaches: TaVer-PerLayer, which fine-tunes all encoder layers, thereby modifying both general and task-specific features, and TaVer, which limits fine-tuning to the final encoder layer to focus on task-specific adaptations while retaining the general features from earlier layers. Both approaches are tested on three target language datasets (Java, Python, and PHP), with the C language dataset serving as the source language. Their performance is assessed using four evaluation metrics: Accuracy, Precision, Recall, and F1-score.

Results. The experimental results in Table 7 show that TaVer consistently outperforms TaVer-PerLayer across all metrics and datasets, supporting the effectiveness of restricting fine-tuning to the final encoder layer. Overall, TaVer achieves an average Accuracy of 52.9%, Precision of 41.79%, Recall of 35.03%, and F1-score of 37.37%, whereas TaVer-PerLayer achieves significantly lower performance, with an average Accuracy of 29.01%, Precision of 19.67%, Recall of 24.68%, and F1-score of 21.65%. For dataset-specific results, TaVer achieves an F1-score of 41.46% on the Java dataset, compared to only 22.45% for TaVer-PerLayer. On the Python and PHP datasets, TaVer also significantly outperforms TaVer-PerLayer, achieving F1-scores of 25.32% and 45.33%, respectively, compared to 12.5% and 30% for TaVer-PerLayer. These results indicate that fine-tuning across all layers, as in TaVer-PerLayer, disrupts the general representations learned in the earlier layers, which are crucial for cross-lingual knowledge transfer. In contrast, restricting fine-tuning to the final layer, as in TaVer, preserves the generality of earlier layers while focusing on task-specific adaptations, leading to superior performance.

Conclusion. Task-specific fine-tuning in the final encoder layer is more effective than fine-tuning across all layers for cross-lingual code vulnerability detection. This strategy enables the model to retain the general features captured by earlier layers while focusing on task-specific adjustments in the later layers. By adopting this design, TaVer

achieves significant improvements across all metrics and datasets, demonstrating its superiority as a cross-lingual transfer learning approach for code vulnerability detection.

6 THREATS TO VALIDITY

This section analyzes the threats to validity in this study from three perspectives: internal validity, external validity, and construct validity. We also discuss the measures taken to mitigate these threats.

6.1 Internal Validity

Threats to internal validity primarily arise from potential biases in the experimental design and the process of replicating approaches. Differences in interpretation or undisclosed implementation details during the replication of comparative approaches may lead to biased results. To mitigate this threat, we strictly adhered to the descriptions provided in the literature when replicating the approaches, and we repeated the experiments multiple times under the same hardware and software environments. Statistical analysis was performed to ensure the consistency and reliability of the results.

6.2 External Validity

External validity concerns the generalizability of the approach across different contexts. Our experiments were conducted using several publicly available datasets, but the representativeness of these datasets may limit the generalizability of the approach. Although we tested the approach on multiple programming languages and under varying conditions, these datasets may not comprehensively cover all real-world scenarios. Future research will further validate the approach's effectiveness on a broader range of datasets and environments to enhance its applicability.

6.3 Construct Validity

Construct validity threats stem from limitations inherent to the approach itself. While TaVer performs well in multilingual environments, its effectiveness may be constrained when dealing with extremely limited data or languages with unique characteristics. Moreover, TaVer's performance is highly dependent on the pre-training and transfer learning process, which is influenced by the quality and diversity of the training data. If the source language data is not sufficiently rich, transfer learning may yield suboptimal results, particularly when there is a significant divergence between the source and target languages. Future research will focus on optimizing these aspects to improve the approach's adaptability and performance further.

In summary, although we have taken measures to reduce the threats to validity, further research is required to explore and refine these aspects to ensure the robustness and generalizability of the approach.

7 CONCLUSION

Code vulnerability detection is essential for ensuring the security of software systems. However, existing approaches face challenges in handling complex temporal information and detecting vulnerabilities in low-resource programming languages, making it difficult to fully capture potential vulnerabilities within code execution paths.

To address the challenges of achieving high accuracy and effective cross-lingual generalization in vulnerability detection for low-resource programming languages, this paper presents a novel approach—TaVer. TaVer comprises two key components: 1) Code Vulnerability Detection Component, which models temporal variations and dependencies in execution paths from the abstract syntax tree, capturing complex features to enhance vulnerability detection accuracy; and 2) Cross-Lingual Transfer Component, which facilitates knowledge transfer from high-resource languages to low-resource languages. By learning temporal features across different languages and efficiently transferring this knowledge, TaVer significantly improves performance in vulnerability detection. Experimental results demonstrate that TaVer outperforms state-of-the-art baseline approaches across several key metrics on three low-resource datasets. Notably, on the PHP dataset, TaVer achieved an Accuracy of 54.95%, surpassing the previous best approach by 23.46%.

In future work, we plan to extend the model's application to cover more programming languages and real-world scenarios, further enriching our training and testing datasets to enhance its generalizability in real-world applications.

ACKNOWLEDGMENT

This work was supported by the National Natural Science Foundation of China (No.62472062), the Dalian Science and technology Innovation Fund project (No.2024JJ12GX022), the Fundamental Research Funds for the Central Universities (No.3132024257).

REFERENCES

- [1] WannaCry ransomware attack. 2020. [Online]. Available: https://en.wikipedia.org/wiki/WannaCry_ransomware_attack.
- [2] Number of common IT security vulnerabilities and exposures (CVEs) worldwide from 2009 to 2024 YTD. [Online]. Available: <https://www.statista.com/statistics/500755/worldwide-common-vulnerabilities-and-exposures/>.
- [3] 5 Key Takeaways from the 2020 OSSRA Report. 2020. [Online]. Available: <https://securityboulevard.com/2020/05/5-key-takeaways-from-the-2020-open-source-security-and-risk-allowbreak-analysis-report/>.
- [4] Flawfinder. [Online]. Available: <https://d Wheeler.com/flawfinder/>.
- [5] RATS. [Online]. Available: <https://code.google.com/archive/p/rough-auditing-tool-for-security/>.
- [6] Cppcheck. [Online]. Available: <https://cppcheck.sourceforge.io/>.
- [7] Checkmarx. [Online]. Available: <https://www.checkmarx.com/>.
- [8] Y. Nong, H. Cai, P. Ye, L. Li, and F. Chen, "Evaluating and comparing memory error vulnerability detectors," *Information and Software Technology*, vol. 137, p. 106614, 2021.
- [9] J. Jang, A. Agrawal, and D. Brumley, "Redebug: finding unpatched code clones in entire os distributions," in *2012 IEEE Symposium on Security and Privacy*, pp. 48–62, IEEE, 2012.
- [10] S. Kim, S. Woo, H. Lee, and H. Oh, "Vuddy: A scalable approach for vulnerable code clone discovery," in *2017 IEEE symposium on security and privacy (SP)*, pp. 595–614, IEEE, 2017.
- [11] Z. Li, D. Zou, S. Xu, H. Jin, H. Qi, and J. Hu, "Vulpecker: an automated vulnerability detection system based on code similarity analysis," in *Proceedings of the 32nd annual conference on computer security applications*, pp. 201–213, 2016.
- [12] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, "Sysevr: A framework for using deep learning to detect software vulnerabilities," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2244–2258, 2021.
- [13] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "Vuldeepecker: A deep learning-based system for vulnerability detection," in *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18–21, 2018*, 2018.
- [14] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated vulnerability detection in source code using deep representation learning," in *2018 17th IEEE international conference on machine learning and applications (ICMLA)*, pp. 757–762, IEEE, 2018.
- [15] Y. Zhou, S. Liu, J. K. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8–14, 2019, Vancouver, BC, Canada* (H. M. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. B. Fox, and R. Garnett, eds.), pp. 10197–10207, 2019.
- [16] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?," *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3280–3296, 2021.
- [17] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, "Deepwukong: Statically detecting software vulnerabilities using deep graph neural network," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 3, pp. 1–33, 2021.
- [18] H. Wang, G. Ye, Z. Tang, S. H. Tan, S. Huang, D. Fang, Y. Feng, L. Bian, and Z. Wang, "Combining graph-based learning with automated data collection for code vulnerability detection," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1943–1958, 2020.
- [19] H. K. Dam, T. Tran, T. Pham, S. W. Ng, J. Grundy, and A. Ghose, "Automatic feature learning for vulnerability prediction," *CoRR*, vol. abs/1708.02368, 2017.
- [20] J. Wu, Z. Qiu, Z. Zeng, R. Xiao, I. Rida, and S. Zhang, "Graph autoencoder anomaly detection for e-commerce application by contextual integrating contrast with reconstruction and complementarity," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 1623–1630, 2024.
- [21] H. Zhang, N. Kumar, S. Wu, C. Wu, J. Wang, and P. Zhang, "Anomaly detection with memory-augmented adversarial autoencoder networks for industry 5.0," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 1952–1962, 2024.
- [22] B. Xie, X. Ma, J. Wu, J. Yang, and H. Fan, "Knowledge graph enhanced heterogeneous graph neural network for fake news detection," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 2826–2837, 2024.
- [23] Z. Yang, L. Xu, and L. Zhao, "Efbh: Collaborative filtering model based on multi-hypergraph encoder," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 2939–2948, 2024.
- [24] W. Gao and Z. Zhao, "Self-supervised multi-source heterogeneous data fusion using encode and decode attention for intelligent medical device communication analysis," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 1318–1325, 2024.
- [25] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "Timesnet: Temporal 2d-variation modeling for general time series analysis," in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1–5, 2023*, OpenReview.net, 2023.
- [26] J. Zhang, Z. Liu, X. Hu, X. Xia, and S. Li, "Vulnerability detection by learning from syntax-based execution paths of code," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4196–4212, 2023.
- [27] Y. Zheng, S. Pujar, B. Lewis, L. Buratti, E. Epstein, B. Yang, J. Laredo, A. Morari, and Z. Su, "D2a: A dataset built for ai-based vulnerability detection methods using differential analysis," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 111–120, IEEE, 2021.
- [28] G. Nikitopoulos, K. Dritsa, P. Louridas, and D. Mitropoulos, "Crossvul: a cross-language vulnerability dataset with commit

- data," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 1565–1569, 2021.
- [29] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, pp. 1536–1547, 2020.
- [30] V.-A. Nguyen, D. Q. Nguyen, V. Nguyen, T. Le, Q. H. Tran, and D. Phung, "Regvd: Revisiting graph neural networks for vulnerability detection," in *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pp. 178–182, 2022.
- [31] W. Hou, H. Zhu, Y. Wang, J. Wang, T. Qin, R. Xu, and T. Shinozaki, "Exploiting adapters for cross-lingual low-resource speech recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 317–329, 2021.
- [32] Code and datasets. 2024. [Online]. Available: <https://github.com/xinyuelong0928/TaVerReferences>.
- [33] S. Neuhaus, T. Zimmermann, C. Holler, and A. Zeller, "Predicting vulnerable software components," in *Proceedings of the 14th ACM conference on Computer and communications security*, pp. 529–540, 2007.
- [34] V. H. Nguyen and L. M. S. Tran, "Predicting vulnerable software components with dependency graphs," in *Proceedings of the 6th international workshop on security measurements and metrics*, pp. 1–8, 2010.
- [35] Y. Shin, A. Meneely, L. Williams, and J. A. Osborne, "Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities," *IEEE Transactions on Software Engineering*, vol. 37, no. 6, pp. 772–787, 2011.
- [36] F. Yamaguchi, K. Rieck, et al., "Vulnerability extrapolation: Assisted discovery of vulnerabilities using machine learning," in *5th USENIX workshop on offensive technologies (WOOT 11)*, 2011.
- [37] F. Yamaguchi, M. Lottmann, and K. Rieck, "Generalized vulnerability extrapolation using abstract syntax trees," in *Proceedings of the 28th annual computer security applications conference*, pp. 359–368, 2012.
- [38] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "Vulcnn: An image-inspired scalable vulnerability detection system," in *Proceedings of the 44th International Conference on Software Engineering*, pp. 2365–2376, 2022.
- [39] S. Cao, X. Sun, L. Bo, R. Wu, B. Li, and C. Tao, "Mvd: memory-related vulnerability detection based on flow-sensitive graph neural networks," in *Proceedings of the 44th international conference on software engineering*, pp. 1456–1468, 2022.
- [40] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pp. 292–303, 2021.
- [41] W. Zheng, Y. Jiang, and X. Su, "Vulspg: Vulnerability detection based on slice property graph representation learning," in *2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*, pp. 457–467, IEEE, 2021.
- [42] W. L. Hamilton, R. Ying, and J. Leskovec, "Representation learning on graphs: Methods and applications," *IEEE Data Eng. Bull.*, vol. 40, no. 3, pp. 52–74, 2017.
- [43] D. Q. NGUYEN, *Representation learning for graph-structured data*. PhD thesis, Monash University, 2021.
- [44] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021.
- [45] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," *Advances in neural information processing systems*, vol. 34, pp. 22419–22430, 2021.
- [46] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?," in *Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, December 8-13 2014, Montreal, Quebec, Canada* (Z. Ghahramani, M. Welling, C. Cortes, N. D. Lawrence, and K. Q. Weinberger, eds.), pp. 3320–3328, 2014.
- [47] D. Luo and X. Wang, "Moderntcn: A modern pure convolution structure for general time series analysis," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*, OpenReview.net, 2024.
- [48] H. Hou and F. R. Yu, "RWKV-TS: beyond traditional recurrent neural network for time series tasks," *CoRR*, vol. abs/2401.09093, 2024.
- [49] T. Zhou, P. Niu, X. Wang, L. Sun, and R. Jin, "One fits all: Power general time series analysis by pretrained LM," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023* (A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, eds.), 2023.
- [50] R. Xu, F. Luo, Z. Zhang, C. Tan, B. Chang, S. Huang, and F. Huang, "Raise a child in large language model: Towards effective and generalizable fine-tuning," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021* (M. Moens, X. Huang, L. Specia, and S. W. Yih, eds.), pp. 9514–9528, Association for Computational Linguistics, 2021.
- [51] D. Guo, A. M. Rush, and Y. Kim, "Parameter-efficient transfer learning with diff pruning," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2021, (Volume 1: Long Papers), Virtual Event, August 1-6, 2021* (C. Zong, F. Xia, W. Li, and R. Navigli, eds.), pp. 4884–4896, Association for Computational Linguistics, 2021.
- [52] Z. Fu, H. Yang, A. M.-C. So, W. Lam, L. Bing, and N. Collier, "On the effectiveness of parameter-efficient fine-tuning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 37, pp. 12799–12807, 2023.



Xinyue Long is an undergraduate student at the School of Information Science and Technology, Dalian Maritime University, expected to graduate in 2026 and pursue advanced studies. Her research interests include intelligent software engineering, transfer learning, and machine learning.



Shikai Guo received the B.S. degree in computer science and technology from Liaoning University, Shenyang, China, in 2011 and the Ph.D. degree in computer applications technology from Dalian Maritime University, Dalian, China, in 2018. He is currently an associate professor at the School of Information Science and Technology, Dalian Maritime University, Dalian, China. His research interests include intelligent software engineering, software testing, and program analysis techniques.



Yu Chai received the B.S. degree in Automation from Dalian Maritime University, Dalian, China, in 2023. He is currently pursuing the M.S. degree at Shanghai Jiao Tong University, Shanghai, China. His research interests include computer vision, transfer learning, and large models.



Hui Li received the Ph.D. degree in computer architecture from Northeastern University, Shenyang, China, in 2013. He is an associate professor at the School of Information Science and Technology, Dalian Maritime University. His research interests include intelligent software engineering, mining software repositories, and recommendation system.



Sumaira Ameer Jan is a graduate student at Dalian Maritime University, Dalian, China. Her research interests include intelligent software engineering and machine learning.



Qian Ma received the Ph.D. degree in computer software and theory from Northeastern University, China, in 2019. She is currently an associate professor at Dalian Maritime University, China. Her research interests include data management and data cleaning.



Qiao Ning received the Ph.D. degree from the School of Information Science and Technology, Northeast Normal University, China, in 2019. She is currently an associate professor at the School of Artificial Intelligence and Computer Science, Jiangnan University. Her research interests include machine learning and bioinformatics.